
GRAPHITEEDGE PRACTICAL GUIDE

The Calm Website Deployment Checklist

A step-by-step launch companion for plain HTML, CSS and JavaScript sites - with extra checks for Vite, GitHub, Netlify and Vercel.

PLAN Know what you are publishing.	TEST Check the live site, not just localhost.	RECORD Leave yourself a trail for the next update.
--	---	--

Web development is not just code - it is understanding how the web works. Once you see the system, you can build anything.

graphitedge.com.au

00 How to use this checklist

Use it before launch, immediately after launch, and again whenever you make a significant update. Tick what applies; skip what does not.

The one rule worth remembering

A site working on your laptop is encouraging. It is not deployment evidence. Always test the live URL from another browser session or device.

Project details

Project or site name	
Your name	
Deployment method	
Hosting service	
Repository URL	
Live URL	
Deployment date	

Your deployment path

- ☐ Traditional hosting or cPanel-style file manager
- ☐ Manual static upload or drag-and-drop hosting
- ☐ GitHub Pages
- ☐ Netlify connected to GitHub
- ☐ Vercel connected to GitHub
- ☐ Another provider: _____

Keep passwords out of this document. Record only safe project and service details.

01 Universal pre-deployment checks

These checks apply no matter where the site will be hosted.

Files and folders

- ☐ The homepage is named index.html.
- ☐ CSS and JavaScript files are linked correctly.
- ☐ Images, fonts and other assets are inside the project folder.
- ☐ File and folder names are simple, consistent and lowercase.
- ☐ There are no accidental spaces or confusing duplicate filenames.
- ☐ No links point to files on my own computer.
- ☐ Unused test files and abandoned drafts have been removed.
- ☐ I know exactly which folder will be uploaded or published.

Content and trust

- ☐ The browser page title is meaningful.
- ☐ The main heading clearly explains the page or site.
- ☐ Placeholder copy, dummy links and sample contact details are gone.
- ☐ Spelling, grammar, dates, prices and contact details have been checked.
- ☐ Navigation labels and calls to action make sense to a first-time visitor.
- ☐ The site makes sense to someone who did not build it.

02 Layout, accessibility and technical checks

Launch is a poor time to discover the mobile menu has wandered off.

Layout and accessibility

- ☐ The site works at desktop, tablet and phone widths.
- ☐ Content does not overflow sideways on a narrow screen.
- ☐ Text is readable and controls are easy to tap.
- ☐ Colour contrast is sufficient for text and controls.
- ☐ Keyboard focus is visible and the site can be navigated without a mouse.
- ☐ Headings follow a logical order.
- ☐ Useful images have useful alt text; decorative images have empty alt text where appropriate.
- ☐ Images resize without distortion.
- ☐ Animation and movement do not distract or block use.

Technical preflight

- ☐ All navigation links work.
- ☐ Images, stylesheets, scripts and fonts load correctly.
- ☐ Buttons, forms and interactive features behave as expected.
- ☐ The browser console shows no obvious errors.
- ☐ At least one link or interaction has been tested on every page.
- ☐ The project has been tested locally before publishing.

03 Choose the correct files to publish

The server should not have to go on a treasure hunt for your homepage.

Plain HTML, CSS and JavaScript

Publish the folder that contains index.html at its top level. Keep linked CSS, JavaScript, images and other assets in the same relative structure.

Correct basic structure

```
my-site/  
index.html  
styles.css  
script.js  
images/
```

- ☐ index.html is at the root of the folder being published.
- ☐ The project folder has not been uploaded one level too deep.
- ☐ All asset folders were included in the upload.
- ☐ Relative paths still match the published folder structure.

Vite projects

A Vite project normally needs a production build. Your source folder is the workshop; the generated output is the version packed for visitors.

Typical commands and output

```
npm run build  
output: dist/
```

- ☐ The project runs locally before building.
- ☐ npm run build completes successfully.
- ☐ The generated dist folder has been inspected.
- ☐ The host is publishing the generated output, not the raw development project.
- ☐ Any routing or base-path requirements have been checked for this project.

04 Deployment method checks

Complete only the section matching your chosen method.

Traditional hosting / file manager

- ☐ I know the public website folder name, such as public_html, htdocs or www.
- ☐ The correct files were uploaded to the correct public folder.
- ☐ index.html is directly inside the public folder.
- ☐ The site was opened and tested after the upload completed.

Manual static upload

- ☐ I selected the correct folder for upload.
- ☐ For a plain site, index.html is at the folder root.
- ☐ For Vite, the build ran first and the generated output was uploaded.
- ☐ I recorded which folder and version were uploaded.

GitHub Pages

- ☐ The current project files are committed and pushed to GitHub.
- ☐ GitHub Pages is enabled for the correct branch and folder.
- ☐ I waited for the publishing workflow to complete.
- ☐ I opened and tested the published GitHub Pages URL.

05 Git-connected deployment checks

For Netlify, Vercel and similar platforms, the build log is your witness statement.

Netlify connected to GitHub

- ☐ Netlify is connected to the correct repository and branch.
- ☐ The build command is correct for this project.
- ☐ The publish directory is correct; for many Vite projects it is dist.
- ☐ Environment variables, redirects or headers are configured if the project needs them.
- ☐ The deployment log was checked if the build failed.
- ☐ The live Netlify URL was tested.

Vercel connected to GitHub

- ☐ Vercel is connected to the correct repository and branch.
- ☐ The detected framework and project root are correct.
- ☐ The build command and output directory match this project.
- ☐ Environment variables are configured if needed.
- ☐ The deployment log was checked if the build failed.
- ☐ The live Vercel URL was tested.

Provider settings change

Treat example settings as a starting point, not a sacred tablet. Confirm the current instructions in your host documentation and in the project configuration.

06 Post-deployment verification

Now test the website as a visitor, not as the person who knows where every loose floorboard is.

- ☐ The live URL opens in a normal browser window.
- ☐ The live URL opens in a private or incognito window.
- ☐ The site opens on another device or network where practical.
- ☐ The homepage and every main page load.
- ☐ Navigation links, buttons and key interactions work.
- ☐ Images, styles, scripts and fonts load on the live site.
- ☐ The mobile layout was checked on the live URL.
- ☐ The browser console was checked on the live site.
- ☐ HTTPS is active and there are no obvious security warnings.
- ☐ Both the provider subdomain and custom domain work as intended, where applicable.
- ☐ Any DNS change has been allowed time to propagate.
- ☐ The most important issue found after launch has been fixed and retested.

Live test notes

Issue found	
Likely cause	
Change made	
Retest result	

07 Continuous improvement log

Deployment is the start of maintenance, not a ceremonial finish line.

A repeatable workflow

Manual workflow

Notice issue -> write it down -> fix locally -> test locally -> upload changed files -> test live -> record the change.

Git-based workflow

Notice issue -> create task -> fix locally -> test -> commit -> push -> deployment runs -> test live -> record the result.

Date	Issue or improvement	Change made	Local?	Live?

08 Final confidence check

A website is ready to share for review when the answers below are honestly “yes”.

- ☐ Someone else can open the website from their own device.
- ☐ The homepage clearly explains what the site is.
- ☐ Visitors can move around the site without dead ends.
- ☐ The most obvious errors have been fixed.
- ☐ I know where the source files and live deployment are managed.
- ☐ I know how to make and publish the next update.
- ☐ I recorded enough information to repeat this process later.

Still finding problems?

That is not failure. That is testing doing its job. Fix the important issues first, record the rest, and redeploy with evidence rather than hope.

Understand the system. Build with confidence.

Continue with the free GraphiteEdge deployment tutorial for hosting choices, Vite builds, DNS, troubleshooting and continuous improvement.

graphitedge.com.au/tutorials/deployments

Created by GraphiteEdge - practical web development education from concept to deployment.